Comandos de Listas

```
evento = ["dia","mes","hora"]
notas = [7, 5.5, 3, 8.7]

evento[0] #"dia", ver posicion 0
len(evento) #3, cantidad de elementos
evento[2]="17h" #renombrar elemento
                ["dia","mes","17h"]

#maximos; minimos; suma total; ordenar
max(notas);  min(notas) #8.7 y 3
#para str por orden alfabetico
max(evento) #"mes", ultimo (m)
min(evento) #"dia", primero (d)
sum(notas) #24.2 =7+5.5+3+8.7
notas.sort() #[3,5.5,7,8.7]menor a myr

#agregar elemento en cierta posicion
evento.insert(2,"year")#(indice, elem)
       #["dia","mes","year","hora"]
evento.append("lugar") #agrega ultimo
       #["dia","mes","hora","lugar"]
evento.extend(notas) #une ambas listas

#revertir; eliminar por elem, por posi
evento.reverse() #["hora","mes","dia"]
#elimina el 1er elemento que coincide
evento.remove("hora") #["dia","mes"]
evento.pop(0) #elimina posi: i = 0
                        ["mes","hora"]
evento.pop() #elimina el ultimo
#inidice de primer elem que coincide
notas.index(5.5) #1, (notas[1]=5.5)

#aplicar funcion a toda una lista
rta = [x**2 for x in notas]
rta = list(map(lambda x: x**2, notas))
```

```
#subcadenas de caracteres (o listas)
palab = "Camino"
palab[1] #"a", un caracter particular
palab[0:3] #"Cam",  [0, 3-1]
palab[:4]  #"Cami", [0, 4-1]
palab[4:]  #"no",   [4, final]

palab[i:j:k] #inicio, final, salto
#por defecto i-j-k = primero-ultimo-1
palab[0:6:2] #"Cmn"
palab[ : :2] #"Cmn"
palab[1::2]  #"aio"

#invertir
palab[-1])  #"o" ultimo
palab[-3:]  #"ino" ultimos tres
palab[::-1] #"onimaC" invertir todo
```

Comandos de Strings

```
#!/usr/bin/env python3

#cantidad de caracteres (+espacios)
len("Buen dia") #8

#separar en una lista; unir una lista
saludo = "Hola a Todos"
saludo.split() #["Hola", "a", "Todos"]
"nota.txt".split(".") #["nota","txt"]
fecha = ["2026","09","18"]
separador = "/" #espacio=" "; vacio=""
separador.join(fecha) #"2026/09/18"
"buen"+" "+"dia" #"buen dia" suma str

#verificar si un string es float o int
pi = str(3.1416)
pi.replace(".","").isdigit() #True
```

Comandos de Matemática

```
abs(x) #valor absoluto de x

10 / 3 # = 3.333 division real
10// 3 # = 3 division entera
10 % 3 # = 1 resto

#redonder con o sin decimal
round(3.77 )  # = 4
round(3.77, 1) #=3.8 un decimal

z = complex(2,3) # 2+3j complejos
z.real # = 2
z.imag # = 3

#----
import math as m

m.sqrt(x) #raiz cuadrada
m.factorial(n) #factorial de n
m.dist([v1,v2], [u1,u2]) #distancia
m.hypot(x,y,z) #norma vector

m.exp(x) #funcion e^x
m.log(x) #logaritmo natural
m.log10(x) #logaritmo base 10

m.sin(x)  #seno
m.asin(x) #arco seno

m.randians(180) # = 3.14
m.degrees(3.14) # = 180

m.pi # = 3.1415
m.e  # = 2.7182
```

Comandos de NumPy

```
import numpy as np

np.array( [Lista] ) # array
np.zeros(tamaño) #todo ceros
np.eye(N) #matriz identidad NxN

Mat.shape # tupla con tamaño de matriz
Mat.reshape(3,9) #nuevo tamaño
Mat.size #cant de elementos

np.sum(Vector) #suma todos sus elem
np.mean(Mat) #promedio
np.argmax(Vector) #indice i del maximo

Mat1 @ Mat2 #producto matrices algebra
Mat.flatten() #convierte en vector
np.fliplr(Mat) #espejo [2,1]->[1,2]
Mat.T #matriz Transpuesta

M[i , :] #fila i-esima de la matriz M
M[: , j] #columna j-esima
#indice periodico, cuando i sale del
M[(i+1) % n, j] #rango, va al inicio

#condicion de igualdad de matrices
np.all( Mat1 == Mat2 ) #True o False

np.arange(min,max,dx) #[min,max),salto
np.linspace(min,max,npuntos)#[min,max]
np.logspace(0,3,4) # [1,10,100,1000]

#distribucion normal (media, varianza,
np.random.normal(0,5,[100,2]) #tamaño)

np.random.rand(tamaño) #entre 0 y 1
np.random.randint(min,max,size)#entero
np.random.uniform(min,max,tamaño)

import scipy.interpolate as sp
fx= sp.CubicSpline(x,y) #interpolacion
b,m=np.polyfit(xdato,ydato,1) #1 grado

np.savetxt("guardar.txt",[vec1,vec2])
cargar = np.loadtxt("guardar.txt")
#x = cargar[0]; y = cargar[1]

np.save("archivo.npy",Vector) #guardar
v2 = np.load("archivo.npy") #cargar
```

Comandos de Graficación

```
import matplotlib.pyplot as plt

#x = array; y1 = fn(x)

#graficar linea continua o puntos
plt.plot(x,y1, label="fn1 continua")
plt.scatter(x,y1,label="datos")#puntos

#nombre de los ejes
plt.title("velocidad vs tiem") #titulo
plt.xlabel("t[s]")
plt.ylabel("v[m/s]")
plt.legend() #muestra labels del plot

#guardar figura
#plt.show() #muestra.No usar en nano
plt.savefig("nombre.png")

#----
"grafico en 3D"
#x=linspace(min,...  y = linspace...
#def fn(x,y): x^2+y^2
z = fn(x,y) #para graficar curva

Xg,Yg = np.meshgrid(x,y) #grilla
Zg = fn(Xg,Yg) #para grafic superficie

plt.subplot(projection="3d")
plt.plot(x,y,z) #opcion 1: una linea
plt.gca().plot_surface(Xg,Yg,Zg) #opc2

#nombres ejes igual que antes, pero
plt.gca().set_zlabel("eje z [m]")
#guardar o mostrar como antes
```